

PROJECT REPORT

Pixel Art Editor

A console-based pixel drawing application developed in C

Course	CSE115
Section	4
Project Group	6
Programming Language	C

Team Member	Student ID
Lamia Nazmul	2625327042
Anindita Barua Sneha	2622449642
Jubair Pasha	2621036642
Nahidul Islam Anim	2622013042
Md. Samiuzzaman Bhuiyan	2622129042

1. Abstract

The Pixel Art Editor is a console application written in C that revolves around a 15 x 15 grid. Each cell in the grid represents a pixel and contains a color value. An empty cell is assigned a value of zero. Users can draw and erase individual pixels, fill connected regions, pick colors from a fixed palette, load an image into the canvas and save the edited result as a PNG. The interface is meant to be interacted with directly on the console. It has mouse support and keyboard controls for the core commands.

This project illustrates fundamental C programming concepts within a practical application. These ideas involve arrays, functions, loops, decision-making statements, boundary checking, input handling, modular design, and menu-driven interaction. It also demonstrates how to map the color of the image to the nearest color in a fixed RGB palette when loading images.

2. Introduction

Pixel art is an image made up of individual blocks or pixels. In this project the concept is simplified into a fixed grid so that each cell can be edited independently. The editor is therefore a small drawing tool, and as a practical exercise in C for the representation of two-dimensional data.

The main idea is to have a simple editing workflow: start the canvas, select a color and tool, select a grid position, perform the operation, and immediately draw the updated canvas. Changes to grid cells are preceded by boundary checks to ensure that user actions are kept within the allowable 15 x 15 area.

2.1 Project Objectives

- Have a two-dimensional array represent a drawable canvas.
- Modularize draw, erase, fill, render, load and save.
- Provide an interactive console interface with mouse supported coordinate selection.
- Use core C programming principles in a complete working application.

3. System Design and Core Features

The editor is organized around a canvas data structure, tool state, palette information, input handling, and rendering. The implementation keeps the canvas bounded by maximum dimensions and initializes each cell to blank before user interaction begins.

3.1 Environment Setup

A 15 x 15 canvas is initialized so that every cell starts with color code 0. Nested loops assign the blank value across all rows and columns. The initialization function also limits the requested width and height to the configured maximum dimensions.

3.2 Color Tool

The set-pixel operation updates a selected grid location only when the coordinates are within the valid canvas range. The active palette color is then stored in that cell.

3.3 Draw and Erase

The draw tool writes the active color to the selected cell. The erase tool resets that cell to palette index 0, returning it to the blank state. Both operations use coordinate validation before modifying the canvas.

3.4 Fill Tool

The flood-fill routine replaces a target color with the selected replacement color and recursively checks the four neighboring directions. It stops when the coordinate is outside the canvas, when the target already equals the replacement, or when the current cell does not match the target color.

3.5 Data Representation

Component	Structure	Role
Canvas	15 x 15 integer grid	0 = blank; non-zero values represent palette colors
Palette	16 fixed colors	Selected by palette index and stored as numeric color codes
Input state	Active tool + active color	Updated from mouse/keyboard

Component	Structure	Role
		interaction
Coordinates	Row/column position	Validated before any canvas modification

4. Interface, Input, and Rendering

4.1 User Interface

The render-user interface function redraws the console view and presents the palette, available tools, current status, coordinate labels, and the 15 x 15 canvas. The display is refreshed after drawing, erasing, or filling so that the user can immediately see the change.

4.2 Mouse Input

The Windows console is configured with mouse input enabled, virtual terminal processing enabled, and a hidden cursor. Mouse coordinates are mapped to palette, tool, or canvas positions. Keyboard shortcuts remain available for save, load, and quit actions.

4.3 File Operations

The editor provides commands to save the current canvas as a PNG and to load an image from the computer. The load and save actions update the interface status so that the user receives feedback about the operation.

4.4 Rendering and Palette Mapping

When an image is loaded, each RGB color is compared with the fixed palette. The nearest palette entry is selected by comparing squared RGB distance, allowing the uploaded image to be represented using the editor palette.

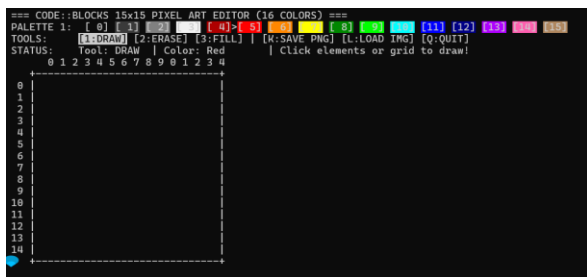


Figure 1: Initial console interface and 15 x 15 Canvas.

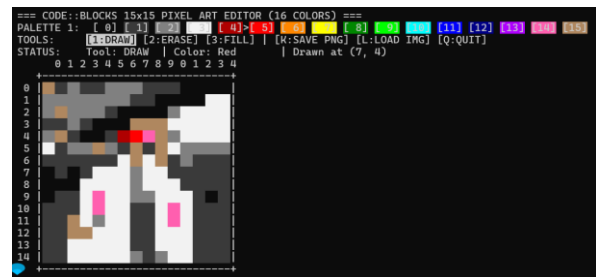


Figure 2: Example of the Canvas after Pixel Edits.

5. Implementation Structure and Member Contribution

The project is divided into functional units so that canvas management, editing tools, interaction, and rendering can be developed and tested separately. The submitted contribution record identifies the following feature ownership.

Feature	Member	Contribution
Environment Setup	Nahidul Islam Anim	Concept, structure, workflow, and canvas coordination.
Color Tool	Anindita Barua Sneha	Selected-pixel color functionality through mouse input.
Draw & Erase Tool	Jubair Pasha	Pixel editing with mouse input and boundary checking.
Fill Tool	Jubair Pasha	Color replacement across matching regions.
User Interface	Anindita Barua Sneha	Interactive menu and tool-selection interface.
Mouse Input	Lamia Nazmul	Mouse coordinate mapping for pixels and controls.
Rendering Tool	Anindita Barua Sneha	Nearest-palette color mapping for loaded images.
Documentation	Md. Samiuzzaman Bhuiyan	Created the final project report.
Testing	Md. Samiuzzaman Bhuiyan	Tested the functionality of the project.

5.1 Overall Program Flow

- Initialize the canvas and console environment.
- Display the palette, tools, status information, and current canvas.
- Receive mouse or keyboard input and map it to an editor action.
- Apply the selected operation after validating coordinates and state.
- Render the updated canvas and continue until the user selects Quit.

6. Testing and Results

The project documentation records a functional test set covering the major user interactions. Every listed scenario was marked as passed.

Test Case	Expected Behavior	Result
Display Canvas	Empty grid shown using spaces	Passed
Coordinate Mapping	Selected pixel coordinates identified through mouse input	Passed
Draw Pixel	Selected pixel updated with the chosen color	Passed
Erase Pixel	Selected pixel reset to blank	Passed
Flood Fill	Matching-color region changed to another color	Passed
Color Choice	Active color changed to a new selection	Passed
Load Image	Selected image uploaded to the canvas	Passed
Save Image	Updated image saved to the computer	Passed
Quit Option	Program terminated successfully	Passed

6.1 Observed Result

According to the submitted test record, the editor handled the tested inputs correctly. Boundary checks behaved as expected, and the draw, erase, fill, load, save, and quit operations completed without reported crashes. The testing outcome indicates that the implemented feature set worked for the documented scenarios.

6.2 Verification Coverage

Area	Coverage
Canvas state	Initialization, coordinate mapping, blank/display behavior
Editing tools	Draw, erase, color selection, fill
Interaction	Mouse-based selection and menu controls
File handling	Image loading and PNG saving
Termination	Quit command and normal program exit

7. Challenges and Solutions

Challenge	Implemented Response
Processing mouse-based color input	Use coordinate mapping to distinguish palette, tool, and canvas positions, then update the relevant UI state.
Preventing invalid grid access	Apply explicit boundary conditions before reading or writing a canvas cell.
Keeping the console grid readable	Use spaces for blank pixels, numeric palette selections, coordinate labels, and a consistent console layout.
Maintaining a stable interactive loop	Keep menu handling, rendering, and input processing in a continuous cycle until the Quit command changes the running state.
Representing loaded images with a fixed palette	Compute RGB distance from the source color to each palette entry and select the nearest palette color.

7.1 Design Strengths

- Simple 15 x 15 data model makes the canvas easy to understand and debug.
- Functions separate initialization, editing, filling, input, rendering, and file operations.
- Boundary checking reduces the risk of invalid grid access.
- The combination of mouse interaction and keyboard shortcuts keeps the application practical in a console environment.

7.2 Limitations Identified in the Submitted Design

The current implementation is intentionally compact. The canvas uses a fixed 15 x 15 size and a fixed palette, so the drawing model is more limited than a full-featured graphics editor. The submitted documentation also identifies additional shape-drawing and canvas-customization capabilities as future improvements.

8. Conclusion and Future Improvements

The Pixel Art Editor successfully demonstrates how core C programming concepts can be assembled into a complete interactive application. The project combines a two-dimensional canvas, modular editing functions, a console user interface, mouse-based coordinate mapping, image loading, and PNG export. The documented test cases all passed, supporting the conclusion that the required feature set was implemented and behaved as expected in the tested scenarios.

8.1 Learning Outcomes

- Using two-dimensional arrays for visual data representation.
- Designing modular functions for initialization, editing, filling, rendering, and input handling.
- Applying loops and conditional statements to practical program logic.
- Implementing boundary checks and user-input validation.
- Building a structured and interactive console-based application.

8.2 Future Improvements

- Add support for drawing different shapes instead of single-pixel edits only.
- Allow the user to customize the canvas size based on the image being uploaded.
- Extend the editing workflow so that larger and more flexible drawing tasks can be supported.